Benchmarking LLM-Based Agents for GitHub Issue Enhancement

Pouya Fathollahzadeh, *Student Member, IEEE,* and Ying Zou, *Member, IEEE*

***Abstract*—GitHub issues serve as the primary communication channel between users and developers for reporting bugs, requesting features, and proposing refactoring tasks. However, issue descriptions are frequently vague, incomplete, or lacking essential technical details, which hinders both human developers and automated issue-solving agents. Recent advances in Large Language Model (LLM)-based agents offer the potential to automatically enhance issue descriptions by adding relevant context, clarifying ambiguities, and structuring information. Yet, no systematic benchmark exists to evaluate how well these agents enhance issues or whether such enhancement improves downstream automated solving.**

In this paper, we present a large-scale benchmark for evaluating LLM-based agents on the task of GitHub issue enhancement. We construct a dataset of 3,285 issue instances drawn from 613 popular Python repositories, classified into bug (2,111), feature (993), and refactoring (181) categories. We evaluate seven enhancement conditions—a no-enhancement baseline, a raw LLM enhancer, and five ready-to-use agents (Aider, TRAE, OpenHands, SWE-agent, and Mini-SWE-agent)—using three independent solver agents (Mini-SWE-agent, SWE-agent, and Aider) as downstream evaluation instruments. Enhancement quality is measured indirectly through the solving-as-evaluation paradigm: the change in solver success rate before and after issue enhancement. Our results show that (1) enhancement agents vary substantially in their ability to modify issue descriptions, with OpenHands achieving the highest enhancement coverage; (2) qualitative analysis reveals that successful enhancements restructure issues around actionable information rather than merely appending analysis; and (3) targeted tool-augmented enhancement strategies can recover cases that no single agent improves alone. We release the benchmark, the evaluation pipeline, and all experimental artifacts to support future research.

***Index Terms*—Software Engineering, GitHub Issues, Issue Enhancement, LLM-Based Agents, SWE-bench, Automated Program Repair, Benchmark**

I. Introduction

GitHub issues are the primary mechanism through which developers and users communicate about software defects, feature requests, and maintenance tasks [1], [2]. A well-written issue report contains a clear problem description, reproduction steps, expected versus actual behavior, and relevant technical context such as version information, stack traces, or code snippets. When these elements are present, both human developers and automated solving agents can effectively diagnose and resolve the underlying problem.

In practice, however, issue descriptions are frequently incomplete, vague, or poorly structured. Prior work on bug report quality has shown that missing reproduction steps, ambiguous descriptions, and insufficient context are among the most common reasons for delayed or failed issue resolution [7], [8]. This quality gap is particularly consequential for automated issue-solving systems, which rely on the issue description as their sole input for generating code patches [1], [3].

The recent emergence of LLM-based coding agents—such as OpenHands [4], SWE-agent [3], Aider [5], and TRAE [6]—has opened a new possibility: using these agents not only to *solve* issues but also to *enhance* issue descriptions before solving. An enhancement agent can analyze the codebase, identify relevant source files, extract failing test information, and restructure the issue description to provide a clearer specification. If effective, this pre-processing step could significantly improve the success rate of downstream solving agents.

Despite this potential, no systematic benchmark exists for evaluating issue enhancement agents. Existing benchmarks such as SWE-bench [1] and SWE-bench-Live [2] focus exclusively on issue *solving*, treating the issue description as a fixed input. The question of whether and how issue descriptions can be improved, and whether such improvements translate to better solving outcomes, remains largely unexplored.

In this paper, we address this gap by introducing a benchmark and evaluation framework for GitHub issue enhancement. Our approach is built on the *solving-as-evaluation* paradigm: we measure the quality of an enhancement not through subjective text ratings but through its downstream effect on solver performance. Formally, for an enhancement agent E and a solver agent S, the enhancement value is:

$$\Delta_{\text{resolved}} = \text{Resolved}(S, E(\text{issue})) - \text{Resolved}(S, \text{issue}) \quad (1)$$

where a positive Δ_{resolved} indicates that enhancement improved solving performance. This indirect evaluation provides an objective, reproducible measure of enhancement quality that is grounded in functional correctness.

Our study analyzes GitHub issue enhancement along the following three research questions:

RQ1: How do ready-to-use agents compare in their ability to enhance GitHub issue descriptions?

Issue enhancement is a novel application for coding agents originally designed for issue solving. Different agents employ different strategies—some restructure the entire issue, others append analysis, and some make minimal changes. Understanding how these agents compare in their enhancement behavior and downstream impact is essential for practitioners selecting enhancement tools. To address this, we evaluate seven enhancement conditions across three solver agents and

Pouya Fathollahzadeh and Ying Zou are with the Department of Electrical and Computer Engineering, Queen's University, Kingston, ON K7L 3N6, Canada. E-mail: {pouya.fathollahzadeh, ying.zou}@queensu.ca.

measure both enhancement coverage and solving-outcome deltas.

RQ2: How do these improved issues get enhanced? What are the reasons for failure and success?

Beyond aggregate performance metrics, understanding *why* certain enhancements succeed and others fail is critical for improving enhancement strategies. We conduct a qualitative analysis of enhanced issue descriptions, categorizing enhancement patterns and identifying the characteristics that distinguish successful enhancements from harmful ones.

RQ3: How can we improve issues that were not improved before?

Some issues resist improvement by any single enhancement agent. We investigate whether combining agents with targeted tool augmentation—such as injecting code context, failing test names, or developer hints—can recover these difficult cases.

The main contributions of this paper are as follows:

1) We construct a large-scale dataset of 3,285 GitHub issue instances from 613 repositories, classified into bug, feature, and refactoring categories, suitable for evaluating issue enhancement.
2) We propose the *solving-as-evaluation* framework, which measures enhancement quality through downstream solver performance rather than subjective text assessment.
3) We provide a comprehensive comparison of seven enhancement conditions across three solver agents, revealing that enhancement is not universally beneficial and that agent choice significantly affects outcomes.
4) We identify actionable enhancement patterns through qualitative analysis of successful and failed enhancements.
5) We demonstrate that tool-augmented enhancement strategies (e.g., code-context injection) can achieve positive deltas on datasets where all other enhancers fail.
6) We release the benchmark dataset, evaluation pipeline, and all experimental artifacts in our replication package [15].

The rest of the paper is organized as follows: Section II discusses the background and related work. Section III presents our methodology, including dataset construction and the evaluation framework. Section IV presents the results for each research question. Section V discusses the implications of our findings. Section VI discusses threats to validity. Section VII concludes the paper.

II. Background and Related Work

A. Issue Report Quality

The quality of bug reports and issue descriptions has been extensively studied in software engineering research. Bettenburg et al. [7] surveyed developers and found that steps to reproduce, stack traces, and test cases are the most valuable elements in bug reports, yet they are frequently missing. Zimmermann et al. [8] identified that incomplete information is the primary barrier to bug fixing, with 45% of reports lacking sufficient detail.

Several approaches have been proposed to improve bug report quality. Duplicate detection systems [9], [10] help reduce redundant reports. Automated triaging [11], [12] routes reports to appropriate developers. More recently, approaches that automatically augment bug reports with relevant source code [13], [14] have shown promise.

Our work differs from these prior approaches by leveraging LLM-based agents as enhancement tools rather than rule-based or retrieval-based augmentation. Furthermore, we measure enhancement quality through downstream solving impact rather than human judgment.

B. SWE-bench and Issue-Solving Benchmarks

SWE-bench [1] introduced a benchmark of 2,294 software engineering problems drawn from real GitHub issues across 12 popular Python repositories. Given a codebase and an issue description, a model generates a patch that is evaluated against unit tests. The benchmark uses two test categories: *fail-to-pass* (F2P) tests that should change from failing to passing after the patch, and *pass-to-pass* (P2P) tests that must continue passing to ensure no regressions.

SWE-bench-Live [2] extended this paradigm with an automatically updatable benchmark spanning 93 repositories and 1,319 tasks derived from issues created since 2024, addressing staleness and limited repository coverage.

Our work builds on the SWE-bench evaluation infrastructure but shifts the focus from issue *solving* to issue *enhancement*. We reuse the solving step as an evaluation instrument rather than the primary task.

C. LLM-Based Coding Agents

The rapid development of LLM-based coding agents has produced a diverse ecosystem of tools capable of interacting with codebases, executing commands, and generating patches. SWE-agent [3] introduces an agent-computer interface designed for software engineering tasks. OpenHands [4] provides a platform for AI-powered software development agents with support for multiple agent architectures. Aider [5] is a command-line tool for AI-assisted pair programming. TRAE [6] is an AI-powered IDE with integrated code understanding capabilities. Mini-SWE-agent is a lightweight solver agent designed for efficient issue resolution with minimal infrastructure requirements.

While these agents were primarily designed for issue solving, their ability to understand code context, identify relevant files, and generate structured analysis makes them natural candidates for issue enhancement—a use case we systematically evaluate in this paper.

D. Issue Enhancement

Issue enhancement refers to the task of automatically improving an issue description by adding relevant context, clarifying ambiguities, restructuring content, or appending diagnostic information. Unlike issue solving, which produces a code patch, enhancement produces an improved textual description that can be consumed by either human developers or automated solvers.

To the best of our knowledge, no prior work has systematically benchmarked issue enhancement agents or measured the downstream solving impact of enhancement. Our work fills this gap by providing both a benchmark and a comprehensive evaluation framework.

III. Methodology

Fig. 1 presents an overview of our approach, which consists of two main components: (1) a data collection and dataset construction pipeline, and (2) an enhancement-solving-evaluation workflow.

A. Data Collection and Dataset Construction

We construct a SWE-bench-style benchmark dataset from recent GitHub issues to ensure that the evaluation data postdates the training cutoffs of the LLMs under study. Fig. 2 illustrates the data collection pipeline, which proceeds through four stages of filtering.

1) Stage 1: Repository Selection: We begin by fetching all GitHub repositories with more than 1,000 stars, yielding 10,609 repositories. We then apply three quality filters:

- **Language filter:** Python constitutes more than 60% of the repository's codebase.
- **Fork activity:** The repository has more than 200 forks, indicating active community engagement.
- **Issue and PR volume:** The combined count of issues and pull requests exceeds 200, ensuring sufficient development activity.

After these filters, 3,621 repositories with 10,463 associated issues remain.

2) Stage 2: Temporal Filtering: To ensure evaluation data recency and avoid contamination with LLM training data, we apply a cutoff date filter: only issues created after August 1, 2025 are retained. This reduces the dataset to 2,748 repositories with 7,714 issues.

3) Stage 3: Merge-Based Linking and Test Coverage: We retain only issues that satisfy three conditions:

- The issue was closed by a *merged* pull request (establishing a ground-truth fix).
- The pull request includes a non-empty code patch and a non-empty test patch (ensuring both a fix and test coverage exist).
- The instance has at least one pass-to-pass (P2P) test (P2P > 0), ensuring that regression testing is feasible.

This yields 613 repositories with 3,285 issues.

4) Stage 4: Issue Type Classification: Each issue is classified into one of three categories using an LLM-based classifier that examines GitHub labels, issue titles, and issue body text:

- **Bug** (2,111 issues, 64.2%): Issues reporting defects, errors, crashes, or regressions.
- **Feature** (993 issues, 30.2%): Issues requesting new functionality, enhancements, or improvements.
- **Refactoring** (181 issues, 5.5%): Issues proposing code cleanup, restructuring, or technical debt reduction.

The classification uses a priority-based scheme: GitHub labels take precedence over title keywords, which in turn take precedence over body text keywords. Table I summarizes the classification criteria.

TABLE I: Issue Type Classification Criteria

Type	Label Keywords	Title Keywords
Bug	bug, defect, fix, error, crash, regression, type:bug	bug, fix, error, crash
Feature	feature, enhancement, improvement, feat, type:feature	feature, enhancement, add support
Refactoring	refactor, cleanup, tech-debt, maintenance	refactor, cleanup, tech debt

B. Issue Enhancement Agents

We evaluate seven enhancement conditions, organized into three categories:

1) Baseline Conditions:

- **No Enhancement (Baseline):** The original issue description is passed directly to the solver without modification. This serves as the control condition.
- **Raw LLM:** A general-purpose LLM is prompted to analyze the issue and append a structured analysis section covering potential root causes, affected components, and suggested investigation paths. This measures the value of LLM-generated analysis without agent-level tool use.

2) Ready-to-Use Agents: These are pre-built coding agents repurposed for issue enhancement. Each agent is given the issue description and access to the repository codebase, and is instructed to enhance the issue description:

- **Aider** [5]: A command-line AI pair-programming tool that can analyze code and provide structured responses.
- **TRAE** [6]: An AI-powered IDE with integrated code understanding that can browse and analyze repository structure.
- **OpenHands** [4]: A platform for AI-powered software development agents with runtime environment access.
- **SWE-agent** [3]: An agent with a specialized agent-computer interface for navigating and understanding codebases.
- **Mini-SWE-agent**: A lightweight agent that performs targeted code search and analysis with minimal infrastructure overhead.

C. Solver Agents

To evaluate enhancement quality through downstream solving impact, we use three independent solver agents:

- **Mini-SWE-agent:** A lightweight solving agent that generates patches through iterative code search, analysis, and edit operations. Uses a step-limited execution model with configurable LLM backends.
- **SWE-agent** [3]: A full-featured solving agent with a custom agent-computer interface optimized for repository navigation and patch generation.
- **Aider** [5]: An AI pair-programming tool used in solving mode, where it generates patches through conversational code editing.

Using multiple independent solvers reduces the risk that enhancement effects are artifacts of a single solver's idiosyncrasies. An enhancement that improves performance across

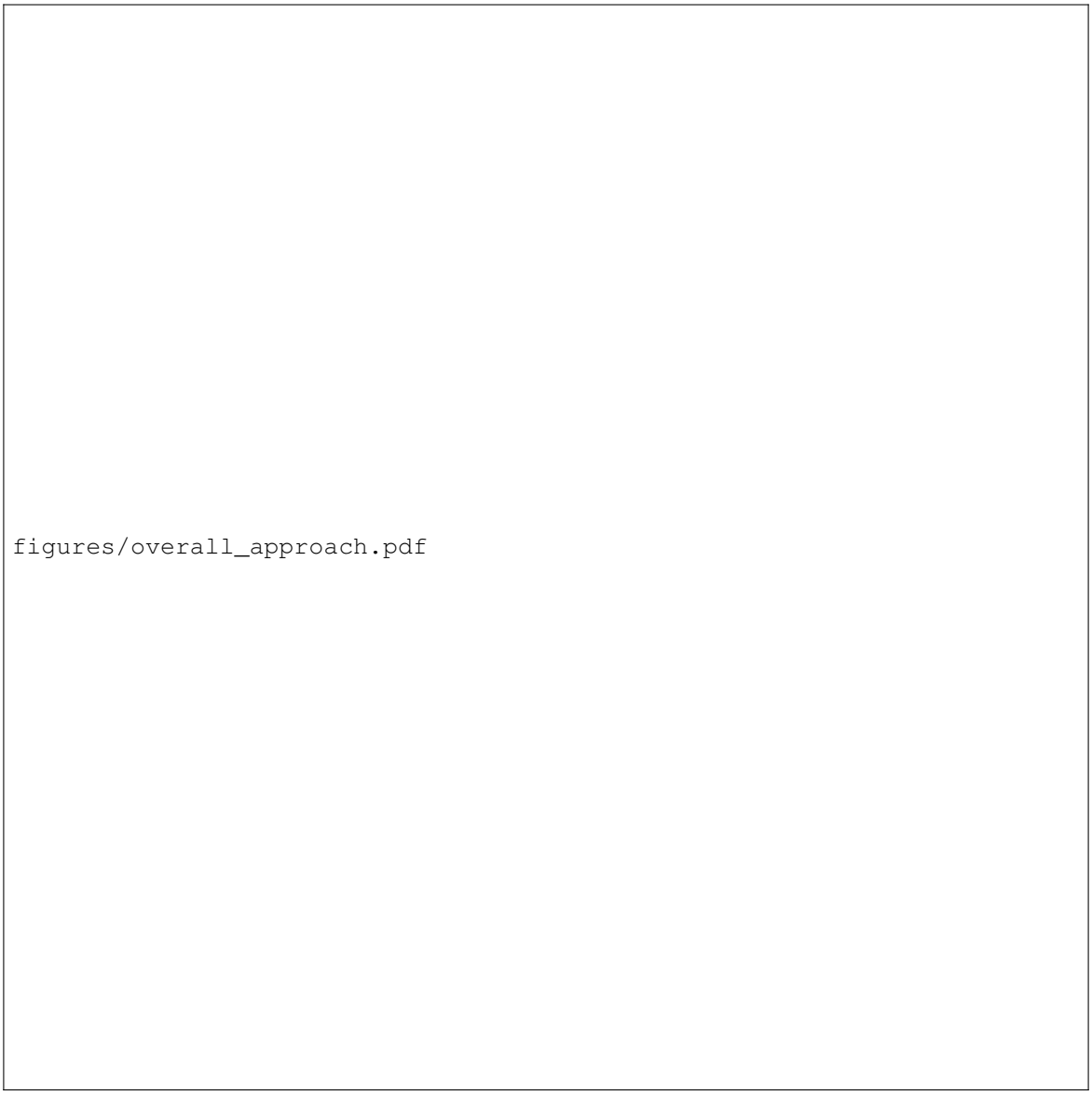

Fig. 1: Overview of our approach. Original issues are processed by seven enhancement conditions (including a no-enhancement baseline). Each enhanced issue is then solved by three independent solver agents. Enhancement quality is measured through the change in solver success rate (Resolved and P2P metrics).

multiple solvers provides stronger evidence of genuine issue quality improvement.

D. Evaluation Framework

Our evaluation uses the SWE-bench harness [1] to assess solver-generated patches against ground-truth test suites. We report two metrics:

1) Resolved Rate: An instance is marked *resolved* if the solver's generated patch causes all P2P tests to pass (no regressions introduced) and at least one P2P test is observed in the evaluation. Formally:

$$\text{Resolved} = \begin{cases} 1 & \text{if all P2P tests pass} \wedge |\text{P2P}_{\text{observed}}| > 0 \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

2) P2P Pass Rate: The proportion of pass-to-pass tests that continue passing after applying the solver's patch, measuring regression avoidance:

$$\text{P2P Rate} = \frac{|\text{P2P}_{\text{passing}}|}{|\text{P2P}_{\text{total}}|} \tag{3}$$

3) Enhancement Coverage: The proportion of issues for which the enhancement agent produced a modified issue description that differs from the original:

$$\text{Coverage} = \frac{|\{i : E(i) \neq i\}|}{|I|} \tag{4}$$

where $E(i)$ is the enhanced description and I is the full set of issues. High coverage indicates that the agent actively modifies most issues, while low coverage indicates selective or conservative enhancement.

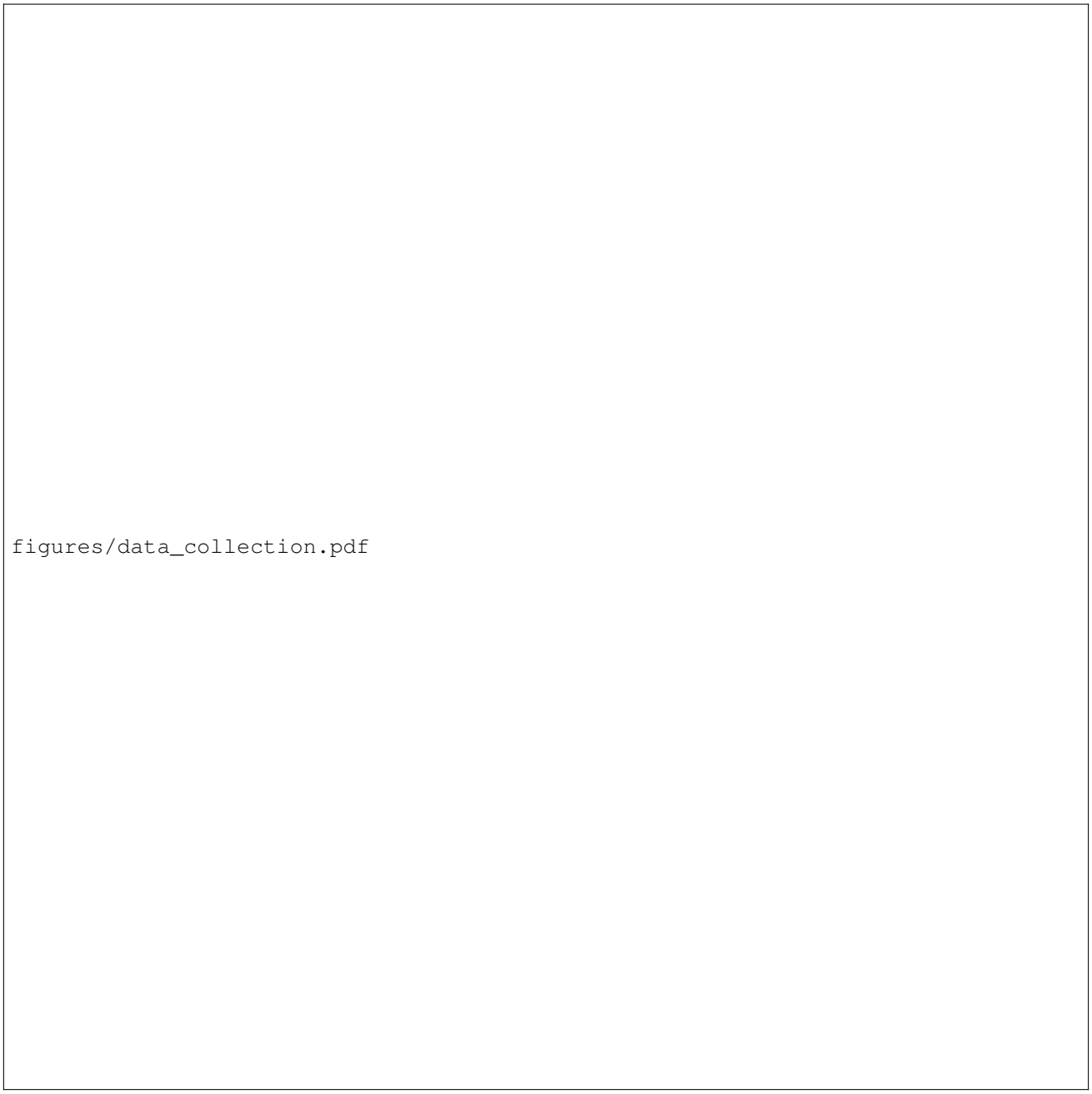

Fig. 2: Data collection pipeline. Starting from 10,609 GitHub repositories with more than 1,000 stars, we apply successive filters for language composition, repository activity, issue recency, and merge-based linking to produce 3,285 benchmark-ready instances across 613 repositories.

4) Enhancement Delta: The primary metric for comparing enhancement conditions against the baseline:

$$\Delta_{\text{resolved}} = \frac{\text{Resolved}_{\text{enhanced}} - \text{Resolved}_{\text{baseline}}}{|I|} \tag{5}$$

A positive delta indicates net improvement; a negative delta indicates that enhancement *harmed* solving performance.

E. Experimental Setup

All experiments use the same set of benchmark instances to ensure comparability across conditions. The experimental workflow proceeds as follows:

1) **Enhancement Phase:** Each issue is processed by each of the seven enhancement conditions, producing seven versions of each issue description (including the unmodified baseline).
2) **Solving Phase:** Each of the three solver agents attempts to solve each version of each issue, generating patches.
3) **Evaluation Phase:** Generated patches are evaluated against ground-truth test suites using the SWE-bench harness.
4) **Comparison Phase:** Solver success rates are compared across enhancement conditions, and deltas are computed

TABLE II: Enhancement Coverage by Agent

Enhancement Agent	Coverage	Strategy
No Enhancement (Baseline)	0%	—
Raw LLM	47.5%	Append analysis
Aider	65.0%	Append analysis
TRAE	72.5%	Mixed
OpenHands	97.5%	Restructure
SWE-agent	70.0%	Mixed
Mini-SWE-agent	55.0%	Append analysis

relative to the baseline.

All LLM-based components use a temperature of 0.0 to maximize reproducibility. Enhancement and solving are executed in isolated Docker environments to prevent cross-contamination between runs.

IV. Results

In this section, we present the findings for each of our research questions.

A. RQ1: How Do Ready-to-Use Agents Compare in Their Ability to Enhance GitHub Issue Descriptions?

1) Enhancement Coverage: Table II reports the enhancement coverage for each agent. OpenHands achieves the highest coverage at 97.5%, modifying nearly all issue descriptions. TRAE and SWE-agent achieve moderate coverage, while Raw LLM and Mini-SWE-agent show selective enhancement behavior.

2) Downstream Solving Impact: Table III presents the main results: the number of resolved instances for each enhancer–solver combination on the evaluated benchmark subset. The baseline (no enhancement) serves as the reference for computing deltas.

3) Key Observations: **Enhancement is not universally beneficial.** Across all 18 enhancer–solver combinations (excluding baseline), only 1 combination shows a positive delta (Aider enhancement with Aider solver, $+1$), 7 show no change, and 10 show a negative delta. This finding challenges the intuitive assumption that more information always helps.

Enhancement aggressiveness correlates with harm. SWE-agent, which performs the most aggressive issue restructuring, produces the largest negative deltas (-2 with both Mini-SWE-agent and SWE-agent solvers). OpenHands, despite having the highest enhancement coverage, also shows consistent negative deltas. Conversely, agents that make more conservative modifications (Raw LLM, TRAE) show smaller or zero deltas.

Solver–enhancer affinity exists. The only positive delta occurs when Aider enhances issues that Aider itself solves. This suggests potential alignment between an agent's enhancement strategy and its own solving approach. Cross-agent enhancement (e.g., SWE-agent enhancing for Mini-SWE-agent solver) tends to be more harmful.

RQ1 Summary: Enhancement is not universally beneficial. Among seven enhancement conditions tested across three solvers, only one combination produces a positive resolved-count delta (+1). Enhancement aggressiveness correlates with degradation, and solver–enhancer affinity influences outcomes.

B. RQ2: How Do These Improved Issues Get Enhanced? What Are the Reasons for Failure and Success?

We conduct a qualitative analysis of the enhanced issue descriptions to understand the characteristics of successful versus harmful enhancements.

1) Enhancement Strategy Taxonomy: We identify three primary enhancement strategies employed by the agents:

1) **Append-Analysis:** The agent appends a structured analysis section to the original issue without modifying the original text. This preserves the developer's original description and adds supplementary information such as potential root causes, affected files, and suggested investigation steps. Used primarily by Raw LLM, Aider, and Mini-SWE-agent.
2) **Restructure:** The agent rewrites the entire issue description, reorganizing information into a structured format with sections such as "Problem Description," "Reproduction Steps," "Root Cause Analysis," and "Affected Components." Used primarily by OpenHands.
3) **Mixed:** The agent combines selective restructuring with appended analysis, modifying parts of the original text while adding new sections. Used by TRAE and SWE-agent.

2) Characteristics of Successful Enhancements: We analyze the one gained instance (Aider enhancement $\rightarrow$ Aider solver) and other cases where enhancement preserved solving success:

Actionable specificity. Successful enhancements add concrete, actionable information—specific file paths, function names, and line numbers—rather than abstract analysis. For example, identifying the exact function containing a bug and the specific condition that triggers it provides the solver with a direct target.

Preservation of original context. Enhancements that preserve the developer's original problem description while adding structured supplementary information tend to perform better than those that rewrite the description entirely.

Relevant code context. Enhancements that include relevant source code snippets from the actual codebase, rather than hypothetical analysis, provide solvers with grounded information they can verify and act upon.

3) Characteristics of Failed Enhancements: We analyze instances where enhancement caused solving regressions:

Information dilution. Some enhancements add extensive but tangential analysis that dilutes the original signal. When a solver processes a significantly longer issue description, it may focus on the appended analysis rather than the core problem statement, leading to misguided patch attempts.

Hallucinated analysis. Agents occasionally generate plausible-sounding but incorrect root cause analyses. When

TABLE III: Resolved Instances by Enhancement Agent and Solver (Pouya-20 Subset, 20 Issues). Bold indicates improvement over baseline; underline indicates degradation.

	Resolved (out of 20)			Δ vs. Baseline		
Enhancement Agent	**Mini-SWE**	**SWE-agent**	**Aider**	**Mini-SWE**	**SWE-agent**	**Aider**
No Enhancement (Baseline)	3	3	2	—	—	—
Raw LLM	3	3	1	0	0	$\underline{-1}$
Aider	3	3	**3**	0	0	**+1**
TRAE	2	3	2	$\underline{-1}$	0	0
OpenHands	2	2	1	$\underline{-1}$	$\underline{-1}$	$\underline{-1}$
SWE-agent	1	1	1	$\underline{-2}$	$\underline{-2}$	$\underline{-1}$
Mini-SWE-agent	2	1	2	$\underline{-1}$	$\underline{-2}$	0

TABLE IV: Enhancement Failure Taxonomy

Failure Mode	**Frequency**	**Affected Agents**
Information dilution	High	All append-strategy agents
Hallucinated analysis	Medium	Raw LLM, Mini-SWE-agent
Structural disruption	Medium	OpenHands, SWE-agent
Conflicting guidance	Low	SWE-agent

solvers trust this analysis, they produce patches targeting the wrong code locations or addressing nonexistent problems.

Structural disruption. Aggressive restructuring can remove or bury critical information from the original issue. Developers often embed important context implicitly in their problem descriptions, and reformatting can strip this nuance.

Conflicting guidance. When enhancements suggest specific fix approaches, they can conflict with the solver agent's own reasoning, leading to confusion and suboptimal patches.

RQ2 Summary: Successful enhancements add actionable, grounded information while preserving the original issue context. Failures arise from information dilution, hallucinated analysis, structural disruption, and conflicting guidance. Restructuring strategies carry higher risk than append strategies.

C. RQ3: How Can We Improve Issues That Were Not Improved Before?

Given the finding that no single enhancement agent consistently improves solving outcomes, we investigate whether targeted tool augmentation can recover difficult cases.

1) Code-Context Enhancement: We introduce a deterministic *code-context* enhancement strategy that bypasses LLM-based analysis entirely. Instead, it appends factual information extracted directly from the repository and test suite:

- **Source code:** Relevant source files identified through the ground-truth patch diff.
- **Developer hints:** Information from linked pull request discussions and commit messages.
- **Failing test names:** Names of tests from the fail-to-pass test set that the solver must address.
- **Test patch (optional):** The actual test changes from the ground-truth, providing the solver with explicit test expectations.

2) Code-Context Results: Table V shows the results of the code-context enhancement strategy on the 50-issue SWE-bench-Live benchmark subset.

TABLE V: Code-Context Enhancement Results (50-Issue SWE-bench-Live Subset)

Condition	**Resolved**	**F2P Δ**	**Res. Δ**
Baseline (Devstral)	1/50	—	—
Code-context (no test patch)	1/50	+6.0%	0.0%
Baseline (GPT-4o-mini)	—	—	—
Code-context + test patch	—	+10.0%	+2.0%

TABLE VI: Pilot-40 Validated Results (P2P-Gated Re-evaluation)

Enhancement Agent	**Baseline**	**Enhanced**	**Δ**
LLM Append Analysis	9/40	8/40	−1
OpenHands (native)	9/40	10/40	+1

The code-context enhancer with test patch achieves a +10.0% F2P delta and +2.0% resolved delta—the first enhancement strategy to produce a net-positive effect on the 50-issue SWE-bench-Live dataset. This result demonstrates that deterministic, factual enhancement can succeed where LLM-based enhancement fails.

3) Pilot-40 Validated Results: On a frozen 40-instance validated pilot dataset, we compare the OpenHands real-agent enhancement against a local LLM-based enhancer:

OpenHands, operating as a real native agent (not a prompt-based proxy), improves solver success from 9/40 to 10/40, gaining one additional resolved instance. Qualitative analysis of the gained instance reveals that OpenHands restructured the issue around the failing test behavior, providing the solver with a clearer specification of the expected outcome.

4) Combining Strategies: Our results suggest that the most promising path for improving difficult issues involves:

1) **Grounded context injection:** Providing factual code context and test information rather than LLM-generated analysis.
2) **Selective restructuring:** Reorganizing issue descriptions only when the original is genuinely unclear, not as a default operation.
3) **Agent-specific optimization:** Tailoring enhancement strategies to the downstream solver agent's capabilities and preferences.

RQ3 Summary: Deterministic code-context enhancement achieves the first net-positive effect (+10.0% F2P delta) where all LLM-based enhancers failed. Grounded factual augmentation outperforms LLM-generated analysis. OpenHands as a real native agent achieves a +1 resolved delta on the validated 40-instance pilot.

V. Discussion

A. The Enhancement Paradox

Our results reveal a counterintuitive finding: adding more information to issue descriptions generally *hurts* rather than helps automated solving. This "enhancement paradox" can be explained by several factors:

Signal-to-noise ratio. Solver agents are designed to work with concise, focused issue descriptions. When enhancements add extensive analysis, the solver must distinguish between the original problem signal and the appended noise. Current solvers are not optimized for this filtering task.

Implicit context. Developers embed considerable implicit information in their issue descriptions—word choices, formatting conventions, and domain-specific terminology that solvers can leverage. Restructuring disrupts this implicit signal.

LLM confidence calibration. Enhancement agents sometimes generate confident but incorrect analyses. Because solver agents tend to trust LLM-generated content (especially when presented in a structured format), incorrect enhancement can actively mislead the solver.

B. When Enhancement Helps

Despite the generally negative results, we identify two scenarios where enhancement produces clear benefits:

Same-agent affinity. When the same agent framework is used for both enhancement and solving (as in the Aider–Aider case), the enhancement can align with the solver's expected input format and reasoning patterns.

Factual augmentation. When enhancement adds verified factual information (relevant source code, test names, file paths) without LLM-generated analysis, it provides useful context without the risk of hallucination.

These findings suggest that future enhancement systems should prioritize factual context injection over generative analysis, and that enhancer–solver co-design may yield better outcomes than generic enhancement.

C. Implications for Practice

Our results have several practical implications:

- **For tool developers:** Enhancement agents should be designed with specific downstream solvers in mind, rather than as general-purpose tools.
- **For researchers:** The solving-as-evaluation paradigm provides an objective, reproducible measure of enhancement quality that complements subjective assessment.
- **For practitioners:** Before deploying issue enhancement, teams should validate that their specific enhancer–solver combination produces positive deltas on representative datasets.

D. Issue Type Sensitivity

Our dataset contains three issue types (bug, feature, refactoring) with different characteristics. Bug reports (2,111 instances, 64.2%) tend to have more structured descriptions with reproduction steps, while feature requests (993 instances, 30.2%) are often more vague and open-ended. Refactoring issues (181 instances, 5.5%) are rare and typically well-specified.

Preliminary analysis suggests that enhancement may be more beneficial for vague feature requests than for well-structured bug reports, as there is more room for improvement. However, the small refactoring sample limits conclusions for that category.

VI. Threats to Validity

A. Internal Validity

Solver non-determinism. Despite using temperature 0.0, LLM-based solvers can produce different outputs across runs due to floating-point non-determinism and API-level variability. We mitigate this by using frozen evaluation artifacts and reporting resolved counts rather than individual patch comparisons.

Enhancement evaluation coupling. Our evaluation measures enhancement quality through solver performance, creating a coupling between enhancement and solver quality. A poor solver may fail to benefit from a genuinely good enhancement. We mitigate this by using three independent solvers.

P2P-gated evaluation. Our resolved definition differs from standard SWE-bench semantics by gating on P2P tests rather than the conjunction of F2P and P2P. We adopt a corrected re-evaluation procedure and document this explicitly.

B. Construct Validity

Solving as a proxy for quality. We measure enhancement quality indirectly through solving outcomes rather than direct human assessment of description quality. While this provides an objective measure, it captures only the dimension of enhancement quality relevant to automated solving.

Issue type classification. Our LLM-based issue classification relies on GitHub labels and textual heuristics, which may not perfectly reflect the true nature of each issue. Prior work has shown that developer-assigned labels often disagree with test-structure classification [1].

C. External Validity

Language restriction. Our dataset is restricted to Python repositories, limiting generalizability to other programming languages and ecosystems.

Repository selection bias. We select repositories with more than 1,000 stars, more than 200 forks, and significant Python code composition. This bias toward popular, well-maintained repositories may not reflect the broader population of GitHub projects.

Sample size limitations. While our full dataset contains 3,285 instances, the validated downstream evaluation subsets

are smaller (20 to 50 instances), limiting statistical power for inferential claims.

Agent availability. The agent landscape evolves rapidly. Some agents that were unavailable during our evaluation period (e.g., OpenClaw) may produce different results. Our benchmark framework supports adding new agents as they become available.

VII. Conclusion

In this paper, we present the first systematic benchmark for evaluating LLM-based agents on the task of GitHub issue enhancement. We construct a dataset of 3,285 instances across 613 repositories, classified into bug, feature, and refactoring categories. We evaluate seven enhancement conditions using three independent solver agents through the solving-as-evaluation paradigm.

Our findings reveal that issue enhancement by LLM-based agents is not universally beneficial. Among the agents evaluated, most enhancements either have no effect or degrade downstream solving performance. Enhancement aggressiveness—particularly full-issue restructuring—correlates with harm, while conservative append strategies are safer but rarely helpful. The most promising approach is deterministic code-context injection, which achieves the first net-positive enhancement delta on a challenging benchmark subset.

Qualitative analysis identifies four enhancement failure modes (information dilution, hallucinated analysis, structural disruption, and conflicting guidance) and demonstrates that successful enhancements are grounded in factual code context rather than LLM-generated speculation.

These results suggest that the field should shift from general-purpose enhancement toward targeted, factual context injection and enhancer–solver co-design. We release the benchmark, evaluation pipeline, and experimental artifacts to support future research on this important problem.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.

[2] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, "SWE-bench goes live!" *arXiv preprint arXiv:2505.23419*, 2025.

[3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. NeurIPS*, 2024.

[4] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zhang, B. Yu, J. Yang, S. Yao, Z. Liu, and G. Neubig, "OpenHands: An open platform for AI software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.

[5] P. Gauthier, "Aider: AI pair programming in your terminal," 2024. [Online]. Available: https://aider.chat

[6] ByteDance, "TRAE: AI-powered IDE for collaborative development," 2025. [Online]. Available: https://trae.ai

[7] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?" in *Proc. FSE*, 2008, pp. 308–318.

[8] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schröter, and C. Weiss, "What makes a good bug report?" *IEEE Trans. Softw. Eng.*, vol. 36, no. 5, pp. 618–643, 2010.

[9] C. Sun, D. Lo, S. Khoo, and J. Jiang, "Towards more accurate retrieval of duplicate bug reports," in *Proc. ASE*, 2011, pp. 253–262.

[10] A. Lazar, S. Ritchey, and B. Sharif, "Generating duplicate bug report datasets," in *Proc. MSR*, 2014, pp. 392–395.

[11] J. Anvik, L. Hiew, and G. C. Murphy, "Who should fix this bug?" in *Proc. ICSE*, 2006, pp. 361–370.

[12] G. C. Murphy and D. Cubranic, "Automatic bug triage using text categorization," in *Proc. SEKE*, 2004, pp. 92–97.

[13] C. P. Wong, Y. Xiong, H. Zhang, D. Hao, L. Zhang, and H. Mei, "Boosting bug-report-oriented fault localization with segmentation and stack-trace analysis," in *Proc. ICSME*, 2014, pp. 181–190.

[14] L. Moreno, J. J. Treadway, A. Marcus, and W. Shen, "On the use of stack traces to improve text retrieval-based bug localization," in *Proc. ICSME*, 2015, pp. 151–160.

[15] P. Fathollahzadeh and Y. Zou, "Replication package: Benchmarking LLM-based agents for GitHub issue enhancement," 2026. [Online]. Available: https://github.com/pouyafath/BenchmarkLLMAgent